

acilis_leaf_gate_train_tflite.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Reconnect GPU

Table of contents

ACLIS Leaf Gate — Tiny Le...
0 — Mount Drive & install...
1 — Config
2 — Load Kaggle leaf / no...
3 — TinyLeafGate model
4 — Dataloaders (ACLIS...
5 — Train (2-phase, same...
6 — Test evaluation (PyT...
7 — Export TinyEngine-fri...
7b — Evaluate deployed L...
8 — STM32F746NG budg...
9 — Next steps on the board
+ Section

ACLIS Leaf Gate — Tiny Leaf / Not-Leaf Model (Colab)

Train a **small binary gate** that runs **before** the existing 5-class PlantVillage disease CNN on the STM32F746G-DISCO:

```
capture → leaf/not-leaf gate → if leaf: disease inference  
→ if not: "Not a leaf — cannot do inference"
```

This notebook mirrors the ACLIS training style (`train_acilis_plantvillage_176.py`) and the export flow in `acilis_plantvillage_176_tflite.ipynb` , but targets a **much smaller** model so it fits in the flash/DRAM headroom left by MCUNet-in3.

Item	Disease model (current)	This leaf gate (target)
Task	5-class disease	2-class leaf / not-leaf
Input	176×176	96×96
Backbone	MCUNet-in3	Custom TinyLeafGate CNN
INT8 flash (est.)	~723 KB	~30-60 KB
Peak SRAM (est.)	~271 KB	~50-80 KB

Cascade tip: run models **sequentially** and **reuse the same activation arena**. Extra peak SRAM ≈ 0 if the gate ≤ 271 KB; **flash is additive** (weights for both models).

0 — Mount Drive & install packages

Same stack as the PlantVillage TFLite notebook (TF 2.20 + ONNX export tools).

```
[ ] from google.colab import drive  
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ] # Install TF 2.20 — compatible with numpy 1.26 AND newer numpy  
!pip install -q tensorflow==2.20.0  
!pip install -q onnx==1.17.0 # pin to avoid ml_dtypes conflict  
!pip install -q onnx-simplifier onnxruntime  
!pip install -q flatbuffers torch torchvision  
!pip install -q scikit-learn matplotlib
```

```
16.0/16.0 MB 79.5 MB/s eta 0:00:00  
Installing build dependencies ... done  
Getting requirements to build wheel ... done  
Preparing metadata (pyproject.toml) ... done  
19.2/19.2 MB 100.2 MB/s eta 0:00:00  
2.7/2.7 MB 112.1 MB/s eta 0:00:00  
Building wheel for onnx-simplifier (pyproject.toml) ... done
```

```
[ ] !pip install -q onnx2tf==1.25.2 --no-deps # reinstall without overwriting deps  
!pip install -q sng4onnx  
!pip install -q onnx-graphsurgeon --index-url https://pypi.ngc.nvidia.com  
!pip install -q simple-onnx-processing-tools  
!pip install -q nvidia-pyindex
```

```
145.6/145.6 kB 5.2 MB/s eta 0:00:00  
440.0/440.0 kB 19.9 MB/s eta 0:00:00  
42.1/42.1 kB 2.4 MB/s eta 0:00:00  
Preparing metadata (setup.py) ... done  
Building wheel for nvidia-pyindex (setup.py) ... done
```

```
[ ] import tensorflow as tf  
import onnx  
import torch  
  
print('✅ TensorFlow :', tf.__version__ )  
print('✅ ONNX       :', onnx.__version__ )  
print('✅ PyTorch      :', torch.__version__ )  
print('✅ CUDA         :', torch.cuda.is_available())
```

```
✅ TensorFlow : 2.20.0  
✅ ONNX       : 1.17.0  
✅ PyTorch      : 2.11.0+cu128  
✅ CUDA         : True
```

1 — Config

Edit paths if your Drive layout differs. Put (or symlink) PlantVillage-style leaf images under `LEAF_DATA_DIR` with `train/val/test` folders — **any disease class counts as leaf**.

Non-leaf images are built automatically from CIFAR-10 (+ optional extras).

```
[ ] import os  
from pathlib import Path  
  
# — Paths —  
DRIVE_ROOT = '/content/drive/MyDrive'  
  
# Kaggle zip you upload to Drive root:  
# MyDrive/leaf_noleaf_dataset.zip  
# Internal layout: Leaf v NonLeaf/{train,val,test}/{Leaf,Non_Leaf}/  
PREBUILT_ZIP = f'{DRIVE_ROOT}/leaf_noleaf_dataset.zip'  
  
# Normalized local dataset used by training cells  
DATASET_DIR = '/content/leaf_noleaf_dataset'  
  
OUTPUT_DIR = f'{DRIVE_ROOT}/leaf_gate_output'  
SAVE_PATH   = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x.pth')  
CHECKPOINT_PATH = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_checkpoint.pth')  
ONNX_PATH     = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x.onnx')  
TFLITE_PATH   = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_full_int8.tflite')  
  
# — Model / training —  
IMAGE_SIZE   = 96  
NUM_CLASSES  = 2  
# ImageFolder sorts alphabetically — leaf=0, not_leaf=1  
CLASSES      = ['leaf', 'not_leaf']  
BATCH_SIZE   = 64  
EPOCHS_HEAD  = 5  
EPOCHS_FULL  = 25  
LR            = 1e-3  
PATIENCE     = 8  
TARGET_ACC   = 0.92  
  
# If DATASET_DIR already normalized, skip unzip
```

```

SKIP_IF_DATASET_EXISTS = True

os.makedirs(OUTPUT_DIR, exist_ok=True)

print('✅ Config ready')
print(f'  PREBUILT ZIP : {PREBUILT_ZIP}')
print(f'  DATASET DIR  : {DATASET_DIR}')
print(f'  OUTPUT DIR   : {OUTPUT_DIR}')
print(f'  IMAGE SIZE   : {IMAGE_SIZE}')
print(f'  CLASSES      : {CLASSES} (index 0=leaf, 1=not leaf)')

```

✅ Config ready
 PREBUILT ZIP : /content/drive/MyDrive/leaf_noleaf_dataset.zip
 DATASET DIR : /content/leaf_noleaf_dataset
 OUTPUT DIR : /content/drive/MyDrive/leaf_gate_output
 IMAGE SIZE : 96
 CLASSES : ['leaf', 'not_leaf'] (index 0=leaf, 1=not_leaf)

2 — Load Kaggle leaf / not-leaf dataset

Upload **leaf_noleaf_dataset.zip** to Google Drive root (MyDrive/).

Expected zip layout (Kaggle):

```

Leaf v NonLeaf/
train/Leaf/ ...
train/Non_Leaf/ ...
val/...
test/...

```

This cell unzips it and normalizes folder names to:

```
/content/leaf_noleaf_dataset/{train,val,test}/{leaf,not_leaf}/
```

so ImageFolder class order is leaf=0, not_leaf=1.

```

import os
import shutil
import time
import zipfile
from pathlib import Path

print('► Dataset cell started', flush=True)

CLASS_ALIASES = {
    # Kaggle / common names → canonical ImageFolder names
    'leaf': 'leaf',
    'Leaf': 'leaf',
    'LEAF': 'leaf',
    'not leaf': 'not leaf',
    'Non_Leaf': 'not leaf',
    'non leaf': 'not leaf',
    'NonLeaf': 'not leaf',
    'noleaf': 'not leaf',
    'NoLeaf': 'not leaf',
    'non-leaf': 'not leaf',
}

def dataset_ready(root):
    for split in ('train', 'val', 'test'):
        for cls in ('leaf', 'not leaf'):
            d = os.path.join(root, split, cls)
            if not os.path.isdir(d):
                return False
            if not any(Path(d).iterdir()):
                return False
    return True

def summarize(root):
    for split in ('train', 'val', 'test'):
        n_leaf = len(os.listdir(os.path.join(root, split, 'leaf')))
        n_not = len(os.listdir(os.path.join(root, split, 'not leaf')))
        print(f' [{split}] leaf={n_leaf} not_leaf={n_not}', flush=True)

def find_split_root(extract_root):
    """Find directory that contains train/ with Leaf or leaf inside."""
    for root, dirs, _ in os.walk(extract_root):
        if 'train' not in dirs:
            continue
        train = os.path.join(root, 'train')
        children = set(os.listdir(train))
        # already canonical
        if 'leaf' in children and 'not_leaf' in children:
            return root
        # Kaggle names
        if any(c in children for c in ('Leaf', 'Non_Leaf', 'NonLeaf', 'noleaf')):
            return root
    return None

def normalize_dataset(src_root, dst_root):
    """Copy/rename to dst root/{split}/{leaf|not_leaf}/."""
    if os.path.isdir(dst_root):
        shutil.rmtree(dst_root)

    for split in ('train', 'val', 'test'):
        split_src = os.path.join(src_root, split)
        if not os.path.isdir(split_src):
            raise FileNotFoundError(f'Missing split folder: {split_src}')

        for name in os.listdir(split_src):
            src_cls = os.path.join(split_src, name)
            if not os.path.isdir(src_cls):
                continue
            canon = CLASS_ALIASES.get(name)
            if canon is None:
                # fuzzy
                key = name.lower().replace('-', '_').replace(' ', '_')
                if key in ('leaf',):
                    canon = 'leaf'
                elif 'non' in key or 'no_leaf' in key or key == 'noleaf':
                    canon = 'not_leaf'
            else:
                print(f' ⚠ skipping unknown class folder: {split}/{name}', flush=True)
                continue
            dst_cls = os.path.join(dst_root, split, canon)
            os.makedirs(dst_cls, exist_ok=True)
            # Move files (faster than copy for local extract)
            for fn in os.listdir(src_cls):
                s = os.path.join(src_cls, fn)
                d = os.path.join(dst_cls, fn)
                if os.path.isfile(s):
                    shutil.move(s, d)

    if not dataset_ready(dst_root):
        raise RuntimeError(f'Normalization failed – check folders under {dst_root}')

# — Fast path: already prepared —
if os.path.exists(OUTPUT_DIR) and os.listdir(OUTPUT_DIR) == ['leaf', 'not_leaf']:
    print('Fast path: dataset already prepared')

```

```

if SKIP_IF_DATASET_EXISTS and dataset_ready(DATASET_DIR):
    print(f'✅ Using existing {DATASET_DIR}', flush=True)
    summarize(DATASET_DIR)
else:
    if not os.path.isfile(PREBUILT_ZIP):
        raise FileNotFoundError(
            f'Zip not found: {PREBUILT_ZIP}\n'
            f'Upload leaf_noleaf_dataset.zip to Google Drive root (MyDrive/).'
        )

    print(f'✅ Found zip: {PREBUILT_ZIP}', flush=True)
    t0 = time.time()

    # Copy zip Drive → local disk first (one sequential read), then unzip locally
    local_zip = '/content/leaf_noleaf_dataset.zip'
    if not os.path.isfile(local_zip) or os.path.getsize(local_zip) != os.path.getsize(PREBUILT_ZIP):
        print(' Copying zip to /content (faster than unzipping from Drive)...', flush=True)
        shutil.copy2(PREBUILT_ZIP, local_zip)
        print(f' Copy done in {time.time()-t0:.0f}s', flush=True)
    else:
        print(' Reusing /content/leaf_noleaf_dataset.zip', flush=True)

    print(' Unzipping (may take several minutes)...', flush=True)
    t1 = time.time()
    extract_tmp = '/content/leaf_noleaf_extract'
    if os.path.isdir(extract_tmp):
        shutil.rmtree(extract_tmp)
    os.makedirs(extract_tmp, exist_ok=True)

    with zipfile.ZipFile(local_zip, 'r') as zf:
        zf.extractall(extract_tmp)
    print(f' Unzip done in {time.time()-t1:.0f}s', flush=True)
    print(' Normalizing folder names (Leaf-leaf, Non_Leaf-not_leaf)...', flush=True)

    src_root = find_split_root(extract_tmp)
    if src_root is None:
        raise RuntimeError(
            'Could not find train/Leaf or train/leaf inside the zip. '
            f'Inspect: {extract_tmp}'
        )
    print(f' Detected dataset root: {src_root}', flush=True)

    normalize_dataset(src_root, DATASET_DIR)
    # cleanup extract tree to free disk
    shutil.rmtree(extract_tmp, ignore_errors=True)

    print(f'✅ Dataset ready at {DATASET_DIR} in {time.time()-t0:.0f}s', flush=True)
    summarize(DATASET_DIR)

    # Sanity: ImageFolder class order
    from torchvision.datasets import ImageFolder
    _probe = ImageFolder(os.path.join(DATASET_DIR, 'train'))
    print('ImageFolder classes:', _probe.classes)
    assert _probe.classes == CLASSES, (
        f'Expected {CLASSES}, got {_probe.classes}. '
        'Class folder names must be exactly Leaf/ and not Leaf/.'
    )
    print('✅ Class index OK: leaf=0, not_leaf=1', flush=True)

```

```

▶ Dataset cell started
✅ Found zip: /content/drive/MyDrive/leaf_noleaf_dataset.zip
Copying zip to /content (faster than unzipping from Drive)...
Copy done in 52s
Unzipping (may take several minutes)...
Unzip done in 31s
Normalizing folder names (Leaf-leaf, Non_Leaf-not_leaf)...
Detected dataset root: /content/leaf_noleaf_extract/Leaf v NonLeaf
✅ Dataset ready at /content/leaf_noleaf_dataset in 84s
[train] leaf=9065 not leaf=8066
[val] leaf=1942 not leaf=1728
[test] leaf=1943 not leaf=1730
ImageFolder classes: ['leaf', 'not leaf']
✅ Class index OK: leaf=0, not leaf=1

```

3 — TinyLeafGate model

A tiny depthwise CNN (25–35k-parameters)-designed to sit in the STM32F746 **flash headroom** (301 KB left after the disease model). INT8 TFLite is expected around ~30–60 KB.

You do **not** retrain MCUNet-in3 — this is a separate gate.

```

print('▶ TinyLeafGate cell started', flush=True)

import torch
import torch.nn as nn
import torch.nn.functional as F

print(f' torch {torch.__version__} cuda={torch.cuda.is_available()}', flush=True)

class DepthwiseSeparable(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.dw = nn.Conv2d(cin, cin, 3, stride=stride, padding=1, groups=cin, bias=False)
        self.bn1 = nn.BatchNorm2d(cin)
        self.pw = nn.Conv2d(cin, cout, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)

    def forward(self, x):
        x = F.relu6(self.bn1(self.dw(x)))
        x = F.relu6(self.bn2(self.pw(x)))
        return x

class TinyLeafGate(nn.Module):
    """Binary leaf / not-leaf classifier for STM32 cascade gate."""
    def __init__(self, num_classes=2):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False), # 96→48
            nn.BatchNorm2d(32),
            nn.ReLU6(inplace=True),
        )
        self.blocks = nn.Sequential(
            DepthwiseSeparable(32, 48, stride=2), # 48→24
            DepthwiseSeparable(48, 64, stride=2), # 24→12
            DepthwiseSeparable(64, 96, stride=2), # 12→6
            DepthwiseSeparable(96, 128, stride=1), # 6→6
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.stem(x)
        x = self.blocks(x)
        return self.head(x)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = TinyLeafGate(NUM_CLASSES).to(DEVICE)

```

```

n_params = sum(p.numel() for p in model.parameters())
n_trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
flash_est_kb = n_params / 1024.0
sram_est_kb = 80.0

print('=' * 65)
print('ACLIS TinyLeafGate - leaf / not-leaf')
print('=' * 65)
print(f'Device      : {DEVICE}')
print(f'Parameters    : {n_params:,}')
print(f'Trainable      : {n_trainable:,}')
print(f'Est. INT8 Flash : {(flash_est_kb:.0f)} KB (weights only)')
print(f'Est. Peak SRAM  : {(sram_est_kb:.0f)} KB (activations; refine after TinyEngine)')
print(f'Input          : {IMAGE_SIZE}{IMAGE_SIZE}')
print('=' * 65)

```

```

▶ TinyLeafGate cell started
torch 2.11.0+cu128 cuda=True
=====
ACLIS TinyLeafGate - leaf / not-leaf
=====
Device      : cuda0
Parameters  : 27,538
Trainable   : 27,538
Est. INT8 Flash : ~27 KB (weights only)
Est. Peak SRAM : ~80 KB (activations; refine after TinyEngine)
Input       : 96x96
=====

```

4 — Dataloaders (ACLIS-style augments, scaled down)

```

from torch.utils.data import DataLoader, WeightedRandomSampler
from torchvision import transforms
from torchvision.datasets import ImageFolder
from PIL import Image

class SafeImageFolder(ImageFolder):
    """Skip corrupt images (same idea as train_aclis_plantvillage_176.py)."""
    def __getitem__(self, index):
        for offset in range(5):
            idx = (index + offset) % len(self.samples)
            path, target = self.samples[idx]
            try:
                img = Image.open(path).convert('RGB')
                if self.transform:
                    img = self.transform(img)
                return img, target
            except Exception:
                continue
        return torch.zeros(3, IMAGE_SIZE, IMAGE_SIZE), 0

train_transform = transforms.Compose([
    # Scale/position jitter so gate works when leaf fills ~50-100% of frame
    # (matches firmware full-frame Resize(96), not a center crop).
    transforms.RandomResizedCrop(
        IMAGE_SIZE, scale=(0.5, 1.0), ratio=(0.9, 1.1)
    ),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.RandomRotation(degrees=30),
    transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3, hue=0.1),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
    transforms.RandomErasing(p=0.15, scale=(0.02, 0.08)),
])

val_transform = transforms.Compose([
    # Full-frame resize - same policy as firmware (no center crop).
    transforms.Resize(IMAGE_SIZE, IMAGE_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

train_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'train'), transform=train_transform)
val_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'val'), transform=val_transform)
test_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'test'), transform=val_transform)

print('Classes (must be not leaf, leaf):', train_dataset.classes)
assert train_dataset.classes == CLASSES, (
    f'Expected {CLASSES}, got {train_dataset.classes}. '
    'Folder names must be exactly not leaf/ and leaf/.'
)

# class balance via WeightedRandomSampler
counts = [0] * NUM_CLASSES
for _, y in train_dataset.samples:
    counts[y] += 1
print('Train counts:', dict(zip(CLASSES, counts)))

weights = [1.0 / counts[y] for _, y in train_dataset.samples]
sampler = WeightedRandomSampler(weights, num_samples=len(weights), replacement=True)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, sampler=sampler,
                           num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                         num_workers=2, pin_memory=True)

class_weights = torch.tensor(
    [sum(counts) / (NUM_CLASSES * c) for c in counts],
    dtype=torch.float32, device=DEVICE
)
print('Class weights:', [c: float(w) for c, w in zip(CLASSES, class_weights)])
loss_fn = nn.CrossEntropyLoss(weight=class_weights)

```

```

Classes (must be not leaf, leaf): ['leaf', 'not_leaf']
Train counts: {'leaf': 9065, 'not_leaf': 8066}
Class weights: {'leaf': 0.9448979496955872, 'not_leaf': 1.0619266033172607}

```

5 — Train (2-phase, same pattern as disease model)

```

def train_one_epoch(model, loader, optimizer):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(images)
    correct += outputs.argmax(-1).eq(labels).sum().item()

```

```

        correct += (outputs.argmax(1) == labels).sum().item()
        total += len(images)
    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        outputs = model(images)
        loss = loss_fn(outputs, labels)
        total_loss += loss.item() * len(images)
        preds = outputs.argmax(1)
        correct += (preds == labels).sum().item()
        total += len(images)
        for p, l in zip(preds, labels):
            class_total[l.item()] += 1
            class_correct[l.item()] += int(p.item() == l.item())

    per_class = {
        CLASSES[i]: (class_correct[i] / class_total[i] if class_total[i] else 0.0)
        for i in range(NUM_CLASSES)
    }
    return total_loss / total, correct / total, per_class

best_val_acc = 0.0
patience_counter = 0

# Phase 1 – short warm-up (all layers; model is tiny / from-scratch)
print('\n' + '=' * 65)
print(f'Phase 1 – warm-up epochs={EPOCHS_HEAD} lr={LR}')
print('=' * 65)
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

for epoch in range(EPOCHS_HEAD):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer)
    va_loss, va_acc, per_class = evaluate(model, val_loader)
    marker = '-'
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        patience_counter = 0
        torch.save(model.state_dict(), SAVE_PATH)
        marker = ' ← saved'
    else:
        patience_counter += 1
    print(f' Epoch {epoch+1:3d}/{EPOCHS_HEAD} | '
          f'train {tr_acc:.3f} | val {va_acc:.3f} | '
          f'leaf={per_class["leaf"]:.3f} not_leaf={per_class["not_leaf"]:.3f}{marker}')

# Phase 2 – cosine fine-tune
print('\n' + '=' * 65)
print(f'Phase 2 – fine-tune epochs={EPOCHS_FULL} lr={LR*0.1}')
print('=' * 65)
optimizer = torch.optim.Adam(model.parameters(), lr=LR * 0.1)
scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer, T_0=10, T_mult=2, eta_min=1e-6
)
patience_counter = 0

for epoch in range(EPOCHS_FULL):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer)
    va_loss, va_acc, per_class = evaluate(model, val_loader)
    scheduler.step()
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        patience_counter = 0
        torch.save(model.state_dict(), SAVE_PATH)
        marker = ' ← saved'
    else:
        patience_counter += 1
        marker = f' (patience {patience_counter}/{PATIENCE})'
    torch.save({
        'epoch': epoch,
        'phase': 2,
        'model state': model.state_dict(),
        'best val acc': best_val_acc,
    }, CHECKPOINT_PATH)
    print(f' Epoch {epoch+1:3d}/{EPOCHS_FULL} | '
          f'train {tr_acc:.3f} | val {va_acc:.3f} | '
          f'leaf={per_class["leaf"]:.3f} not_leaf={per_class["not_leaf"]:.3f}{marker}')
    if patience_counter >= PATIENCE:
        print(f'\n Early stopping at epoch {epoch+1}')
        break

print(f'\nBest val accuracy: {best_val_acc:.4f}')

```

```

=====
Phase 1 – warm-up epochs=5 lr=0.001
=====
Epoch 1/5 | train 0.875 | val 0.968 | leaf=0.991 not leaf=0.943 ← saved
Epoch 2/5 | train 0.932 | val 0.976 | leaf=0.993 not leaf=0.957 ← saved
Epoch 3/5 | train 0.941 | val 0.983 | leaf=0.993 not leaf=0.972 ← saved
Epoch 4/5 | train 0.947 | val 0.974 | leaf=0.995 not leaf=0.950
Epoch 5/5 | train 0.948 | val 0.988 | leaf=0.991 not leaf=0.986 ← saved

=====
Phase 2 – fine-tune epochs=25 lr=0.0001
=====
Epoch 1/25 | train 0.960 | val 0.988 | leaf=0.990 not leaf=0.986 (patience 1/8)
Epoch 2/25 | train 0.960 | val 0.989 | leaf=0.991 not leaf=0.986 ← saved
Epoch 3/25 | train 0.960 | val 0.987 | leaf=0.991 not leaf=0.982 (patience 1/8)
Epoch 4/25 | train 0.963 | val 0.989 | leaf=0.992 not leaf=0.984 (patience 2/8)
Epoch 5/25 | train 0.963 | val 0.988 | leaf=0.993 not leaf=0.981 (patience 3/8)
Epoch 6/25 | train 0.964 | val 0.989 | leaf=0.994 not leaf=0.984 ← saved
Epoch 7/25 | train 0.967 | val 0.988 | leaf=0.992 not leaf=0.984 (patience 1/8)
Epoch 8/25 | train 0.967 | val 0.987 | leaf=0.992 not leaf=0.983 (patience 2/8)
Epoch 9/25 | train 0.961 | val 0.989 | leaf=0.991 not leaf=0.986 (patience 3/8)
Epoch 10/25 | train 0.963 | val 0.987 | leaf=0.990 not leaf=0.984 (patience 4/8)
Epoch 11/25 | train 0.966 | val 0.989 | leaf=0.992 not leaf=0.985 (patience 5/8)
Epoch 12/25 | train 0.962 | val 0.989 | leaf=0.992 not leaf=0.986 (patience 6/8)
Epoch 13/25 | train 0.964 | val 0.989 | leaf=0.992 not leaf=0.985 (patience 7/8)
Epoch 14/25 | train 0.968 | val 0.989 | leaf=0.994 not leaf=0.984 ← saved
Epoch 15/25 | train 0.966 | val 0.990 | leaf=0.994 not leaf=0.985 ← saved
Epoch 16/25 | train 0.967 | val 0.990 | leaf=0.994 not leaf=0.984 (patience 1/8)
Epoch 17/25 | train 0.967 | val 0.989 | leaf=0.993 not leaf=0.986 (patience 2/8)
Epoch 18/25 | train 0.965 | val 0.989 | leaf=0.990 not leaf=0.988 (patience 3/8)
Epoch 19/25 | train 0.964 | val 0.989 | leaf=0.993 not leaf=0.986 (patience 4/8)
Epoch 20/25 | train 0.967 | val 0.990 | leaf=0.994 not leaf=0.986 ← saved
Epoch 21/25 | train 0.967 | val 0.991 | leaf=0.993 not leaf=0.989 ← saved
Epoch 22/25 | train 0.971 | val 0.991 | leaf=0.994 not leaf=0.987 (patience 1/8)
Epoch 23/25 | train 0.967 | val 0.990 | leaf=0.992 not leaf=0.987 (patience 2/8)
Epoch 24/25 | train 0.968 | val 0.990 | leaf=0.993 not leaf=0.987 (patience 3/8)
Epoch 25/25 | train 0.968 | val 0.990 | leaf=0.995 not leaf=0.986 (patience 4/8)

Best val accuracy: 0.9913

```

6 — Test evaluation (PyTorch FP32)

This is the **training checkpoint** accuracy, not the MCU model. After INT8 export, run **\$7b** on `accli_leaf_gate_96x_full_int8.tflite` — that is the number that matters for deployment.

```

model.load_state_dict(torch.load(SAVE_PATH, map_location=DEVICE))
te_loss, te_acc, per_class = evaluate(model, test_loader)
print('=' * 65)
print('Final TEST results')
print('=' * 65)
print(f' Test accuracy : {te_acc:.4f} ({te_acc*100:.1f}%)')
print(f' Target      : >={TARGET_ACC*100:.0f}% '
      f'{"PASS ✓" if te_acc >= TARGET_ACC else "below target - add field negatives / train longer"}')
for cls, acc in per_class.items():
    print(f'      {cls:<10} {acc:.3f} ({acc*100:.1f}%)')
print(f' Weights saved : {SAVE_PATH}')
print('=' * 65)

```

Final TEST results

```

=====
Test accuracy : 0.9910 (99.1%)
Target      : >92% PASS ✓
leaf        0.993 (99.3%)
not leaf    0.988 (98.8%)
Weights saved : /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x.pth
=====

```

7 — Export TinyEngine-friendly INT8 TFLite

Do not use ONNX→onnx2tf for this model. That path previously produced a broken graph:

MEAN [1,1,1,128] → TRANSPOSE → RESHAPE [1,1] → FC (channels collapsed — TinyEngine fails)

This cell uses a Keras twin with a TinyEngine-friendly head:

GlobalAveragePooling2D(keepdims=True) → Conv2D(2, kernel 1x1)

That matches how MCUNet/TinyEngine expect GAP + classifier (avg pool + 1x1 conv), with no broken Flatten/Reshape.

```

print('▶ TinyEngine-friendly export started', flush=True)

import numpy as np
import tensorflow as tf
import torch

assert 'model' in dir() and 'SAVE_PATH' in dir(), 'Run training cells first'
model.load_state_dict(torch.load(SAVE_PATH, map_location='cpu'))
model.cpu().eval()
sd = model.state_dict()

def pt_conv_to_keras(w):
    # PT OIHW → Keras HWIO
    return w.detach().cpu().numpy().transpose(2, 3, 1, 0)

def pt_dw_to_keras(w):
    # PT [C,L,Kh,Kw] → Keras [Kh,Kw,C,1]
    return w.detach().cpu().numpy().transpose(2, 3, 0, 1)

def pt_linear_to_conv1x1(w):
    # PT Linear [out,in] → Keras Conv2D [1,1,in,out]
    arr = w.detach().cpu().numpy().T # [in,out]
    return arr.reshape(1, 1, arr.shape[0], arr.shape[1])

def keras_bn_weights(prefix):
    # Keras BN order: gamma, beta, mean, var
    return [
        sd[f'{prefix}.weight'].detach().cpu().numpy(),
        sd[f'{prefix}.bias'].detach().cpu().numpy(),
        sd[f'{prefix}.running_mean'].detach().cpu().numpy(),
        sd[f'{prefix}.running_var'].detach().cpu().numpy(),
    ]

def pt_matched_bn(name):
    # Match PyTorch BatchNorm2d defaults at inference (eps=1e-5).
    # Keras momentum is inverted vs PT: PT momentum=0.1 → Keras momentum=0.9.
    return tf.keras.layers.BatchNormalization(
        epsilon=1e-5, momentum=0.9, name=name)

def conv_pt_pad(x, layer, name):
    """PyTorch padding=1 is 1px on ALL sides. Keras 'same' is NOT that for
    stride=2 on even spatial sizes (often pads 0+1 asymmetrically) — that alone
    caused ~3-5 logit PT-Keras mismatch. Mirror PT with ZeroPadding2D + valid."""
    x = tf.keras.layers.ZeroPadding2D(1, name=f'{name}_pad')(x)
    return layer(x)

def build_keras_twin():
    """NHWc twin: same topology+padding as PT, GAP(keepdims)+1x1 Conv head."""
    def ds_block(x, cout, stride, name):
        x = conv_pt_pad(
            x,
            tf.keras.layers.DepthwiseConv2D(
                3, strides=stride, padding='valid', use_bias=False, name=f'{name}_dw'),
            f'{name}_dw',
        )
        x = pt_matched_bn(f'{name}_bn1')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu1')(x)
        x = tf.keras.layers.Conv2D(
            cout, 1, use_bias=False, name=f'{name}_pw')(x)
        x = pt_matched_bn(f'{name}_bn2')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu2')(x)
        return x

    inp = tf.keras.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3), name='input')
    x = conv_pt_pad(
        inp,
        tf.keras.layers.Conv2D(
            32, 3, strides=2, padding='valid', use_bias=False, name='stem_conv'),
        'stem',
    )
    x = pt_matched_bn('stem_bn')(x)
    x = tf.keras.layers.ReLU(max_value=6.0, name='stem_relu')(x)
    x = ds_block(x, 48, 2, 'b0')
    x = ds_block(x, 64, 2, 'b1')
    x = ds_block(x, 96, 2, 'b2')
    x = ds_block(x, 128, 1, 'b3')
    # keepdims=True → [8,1,1,128] then 1x1 conv → [8,1,1,2] (no Flatten)
    x = tf.keras.layers.GlobalAveragePooling2D(keepdims=True, name='gap')(x)
    x = tf.keras.layers.Conv2D(NUM_CLASSES, 1, use_bias=True, name='cls_conv')(x)
    out = tf.keras.layers.Reshape((NUM_CLASSES,), name='output')(x)
    return tf.keras.Model(inp, out, name='TinyLeafGate_TE')

kmodel = build_keras_twin()

# --- copy PyTorch weights ---
kmodel.get_layer('stem_conv').set_weights([pt_conv_to_keras(sd['stem.0.weight'])])
kmodel.get_layer('stem_bn').set_weights(keras_bn_weights('stem.1'))

# blocks.0 .. blocks.3 → DepthwiseSeparable: dw, bn1, pw, bn2
block_map = [
    ('b0', 'blocks.0'),
    ('b1', 'blocks.1'),
    ('b2', 'blocks.2'),
    ('b3', 'blocks.3'),
]

```

```

for kname, ptname in block_map:
    kmodel.get_layer(f'{kname}.dw').set_weights([pt_dw.to_keras(sd[f'{ptname}.dw.weight'])])
    kmodel.get_layer(f'{kname}.bn1').set_weights(keras_bn_weights(f'{ptname}.bn1'))
    kmodel.get_layer(f'{kname}.pw').set_weights([pt_conv.to_keras(sd[f'{ptname}.pw.weight'])])
    kmodel.get_layer(f'{kname}.bn2').set_weights(keras_bn_weights(f'{ptname}.bn2'))

# head: AdaptiveAvgPool + Flatten + Dropout + Linear(128,2)
# Linear - 1x1 Conv2D
lin_w = sd['head.3.weight'] # [2,128]
lin_b = sd['head.3.bias'] # [2]
kmodel.get_layer('cls_conv').set_weights([
    pt_linear.to_conv1x1(lin_w),
    lin_b.detach().cpu().numpy(),
])

print('✅ Keras twin built and weights copied')
kmodel.summary()

# --- float sanity: PT vs Keras on a few val images ---
kmodel.trainable = False
n_check = min(16, len(val_dataset))
diffs = []
for i in range(n_check):
    img, _ = val_dataset[i] # CHW torch tensor normalized
    # PT
    with torch.no_grad():
        pt_out = model(img.unsqueeze(0)).numpy().reshape(-1)
    # Keras NHWC
    x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
    k_out = kmodel.predict(x, verbose=0).reshape(-1)
    diffs.append(np.max(np.abs(pt_out - k_out)))

print(f'Float max|PT-Keras| over {n_check} images: mean={np.mean(diffs):.5f} max={np.max(diffs):.5f}')
if np.max(diffs) > 0.01:
    raise RuntimeError(
        f'PT-Keras logit mismatch too large (max={np.max(diffs):.5f}). '
        'Do not export TFLite - check BN epsilon=1e-5 and ZeroPadding2D(1)+valid '
        '(not Keras padding=same) to match PyTorch padding=1.'
    )
print('✅ PT - Keras logits match closely')

```

b0_relu1 (ReLU)	(None, 24, 24, 32)	0
b0_pw (Conv2D)	(None, 24, 24, 48)	1,536
b0_bn2 (BatchNormalization)	(None, 24, 24, 48)	192
b0_relu2 (ReLU)	(None, 24, 24, 48)	0
b1_dw_pad (ZeroPadding2D)	(None, 28, 28, 48)	0
b1_dw (DepthwiseConv2D)	(None, 12, 12, 48)	432
b1_bn1 (BatchNormalization)	(None, 12, 12, 48)	192
b1_relu1 (ReLU)	(None, 12, 12, 48)	0
b1_pw (Conv2D)	(None, 12, 12, 64)	3,672
b1_bn2 (BatchNormalization)	(None, 12, 12, 64)	256
b1_relu2 (ReLU)	(None, 12, 12, 64)	0
b2_dw_pad (ZeroPadding2D)	(None, 14, 14, 64)	0
b2_dw (DepthwiseConv2D)	(None, 6, 6, 64)	576
b2_bn1 (BatchNormalization)	(None, 6, 6, 64)	256
b2_relu1 (ReLU)	(None, 6, 6, 64)	0
b2_pw (Conv2D)	(None, 6, 6, 96)	6,144
b2_bn2 (BatchNormalization)	(None, 6, 6, 96)	384
b2_relu2 (ReLU)	(None, 6, 6, 96)	0
b3_dw_pad (ZeroPadding2D)	(None, 8, 8, 96)	0
b3_dw (DepthwiseConv2D)	(None, 6, 6, 96)	864
b3_bn1 (BatchNormalization)	(None, 6, 6, 96)	384
b3_relu1 (ReLU)	(None, 6, 6, 96)	0
b3_pw (Conv2D)	(None, 6, 6, 128)	12,288
b3_bn2 (BatchNormalization)	(None, 6, 6, 128)	512
b3_relu2 (ReLU)	(None, 6, 6, 128)	0
gap (GlobalAveragePooling2D)	(None, 1, 1, 128)	0
cls_conv (Conv2D)	(None, 1, 1, 2)	256
output (Reshape)	(None, 2)	0

Total params: 78,784 (112.32 KB)
 Trainable params: 27,536 (107.57 KB)
 Non-trainable params: 1,216 (4.75 KB)
 Float max|PT-Keras| over 16 images: mean=0.00000 max=0.00000
 ✅ PT - Keras logits match closely

```

print('▶ INT8 TFLite conversion + graph check', flush=True)

import os
from pathlib import Path

calib_ds = val_dataset # already ImageFolder-like

def make_rep_data():
    n = min(200, len(calib_ds))
    for i in range(n):
        img, _ = calib_ds[i]
        x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
        yield x

# Export SavedModel for debugging with Keras 3+
TF_SAVED = '/content/aclics/leaf_gate_tf'
!rm -rf {TF_SAVED}
kmodel.export(TF_SAVED)
print('SavedModel:', TF_SAVED)

converter = tf.lite.TFLiteConverter.from_keras_model(kmodel)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = make_rep_data
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_model = converter.convert()

os.makedirs(os.path.dirname(TFLITE_PATH) or '.', exist_ok=True)
with open(TFLITE_PATH, 'wb') as f:
    f.write(tflite_model)

tflite_kb = os.path.getsize(TFLITE_PATH) / 1024.0
print(f'✅ INT8 TFLite saved: (TFLITE_PATH)')
print(f'File size: {tflite_kb:.1f} KB')

# Also copy next to notebook outputs on Drive root helper

```

```
drive_copy = os.path.join(root_dir, 'aclis_leaf_gate_96x_full_int8.tflite')
with open(drive_copy, 'wb') as f:
    f.write(tflite_model)
print(f'✅ Drive copy: {drive_copy}')
```

```
* Endpoint 'serve'
args 0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 96, 96, 3), dtype=tf.float32, name='input')
Output Type:
TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
```

```
133018992688784: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863112794: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863115888: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018992688816: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018992688976: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863108944: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863112016: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863114794: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863111248: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863110908: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863111448: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863108560: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863112976: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863114896: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863114512: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863110488: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863108176: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863113744: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863110672: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863110096: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863111056: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863112208: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863111632: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863112408: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863115472: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863111824: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863115664: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863113552: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863115280: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863116432: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863115956: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863116848: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863116240: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863109904: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863117392: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863116816: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863117008: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863117208: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863110208: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863118352: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863117776: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863117968: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863118160: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018992688784: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863118544: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863118736: TensorSpec(shape=(), dtype=tf.resource, name=None)
133018863114320: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/convert.py:863: UserWarning: Statistics for quantized inputs were expected, but not specified; continuing without them.
warnings.warn()
```

```
✅ INT8 TFLite saved: /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
```

```
File size: 50.9 KB
```

```
✅ Drive copy: /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
```

```
print('▶ Validate TFLite graph for TinyEngine', flush=True)
```

```
import numpy as np
from tensorflow.lite.python import schema_py_generated as schema_fb
```

```
interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
```

```
interp.allocate_tensors()
```

```
inp = interp.get_input_details()[0]
```

```
out = interp.get_output_details()[0]
```

```
print('Input:', list(inp['shape']), inp['dtype'], 'quant=', inp.get('quantization'))
```

```
print('Output:', list(out['shape']), out['dtype'], 'quant=', out.get('quantization'))
```

```
# Opcode name helper
```

```
_OP_NAMES = {
    getattr(schema_fb.BuiltinOperator, k): k
    for k in dir(schema_fb.BuiltinOperator)
    if k.isupper() and isinstance(getattr(schema_fb.BuiltinOperator, k), int)
}
```

```
buf = open(TFLITE_PATH, 'rb').read()
```

```
fb_model = schema_fb.Model.GetRootAsModel(buf, 0)
```

```
g = fb_model.Subgraphs(0)
```

```
ops = []
```

```
bad = []
```

```
for i in range(g.OperatorsLength()):
```

```
    op = g.Operators(i)
```

```
    oc = fb_model.OperatorCodes(op.OpcodeIndex())
```

```
    code = oc.BuiltinCode()
```

```
    # TFLite v3+ may store code in deprecated field
```

```
    if code == 0 and hasattr(oc, 'DeprecatedBuiltinCode'):
```

```
        try:
```

```
            dep = oc.DeprecatedBuiltinCode()
```

```
            if dep:
```

```
                code = dep
```

```
        except Exception:
```

```
            pass
```

```
        name = _OP_NAMES.get(code, f'OP_{code}')
```

```
        ops.append(name)
```

```
    if name == 'RESHAPE':
```

```
        out_ti = op.Outputs(0)
```

```
        t = g.Tensors(out_ti)
```

```
        shape = [int(t.Shape(j)) for j in range(t.ShapeLength())]
```

```
        # Broken export collapsed 128-D features to scalar-ish [1,1]
```

```
        if shape in ([1, 1], [1]):
```

```
            bad.append(f'RESHAPE → {shape} (collapsed features - TinyEngine will fail)')
```

```
        else:
```

```
            print(f' reshape ok: {shape}')
```

```
print('\nOperators:')
```

```
for i, n in enumerate(ops):
```

```
    print(f' {i:02d} {n}')
```

```
has_pool = any(n in ('MEAN', 'AVERAGE_POOL_2D') for n in ops)
```

```
has_cls = any(n in ('CONV_2D', 'FULLY_CONNECTED') for n in ops)
```

```
print('\nHas GAP/MEAN/AVG POOL:', has_pool)
```

```
print('Has CONV/FC classifier:', has_cls)
```

```
# Quick int8 inference smoke test
```

```
x = np.zeros(tuple(inp['shape']), dtype=np.int8)
```

```
interp.set_tensor(inp['index'], x)
```

```
interp.invoke()
```

```
y = interp.get_tensor(out['index'])
```

```
print('Smoke output:', y)
```

```
if bad or not has_pool or not has_cls:
```

```
    print('❌ Graph not TinyEngine-friendly:')
```

```
    for b in bad:
```

```
        print(' -', b)
```

```
    if not has_pool:
```

```
        print(' - missing pooling/MEAN')
```

```
    if not has_cls:
```

```
print(' - missing CONV/FC')
raise RuntimeError('Fix export before downloading TFLite')
else:
    print('\n✅ TFLite looks TinyEngine-friendly (no broken [1,1] reshape)')
    print('Download:', TFLITE_PATH)
    print('Also on Drive:', os.path.join(OUTPUT_DIR, 'aclis_leaf_gate_96x_full_int8.tflite'))

▶ Validate TFLite graph for TinyEngine
Input: [np.int32(1), np.int32(96), np.int32(96), np.int32(3)] <class 'numpy.int8'> quant= (0.01845697872340679, -16)
Output: [np.int32(1), np.int32(2)] <class 'numpy.int8'> quant= (0.0731718167662605, 9)
    reshape ok: [1, 2]
\nOperators:
00 PAD
01 CONV_2D
02 PAD
03 DEPTHWISE_CONV_2D
04 CONV_2D
05 PAD
06 DEPTHWISE_CONV_2D
07 CONV_2D
08 PAD
09 DEPTHWISE_CONV_2D
10 CONV_2D
11 DEPTHWISE_CONV_2D
12 CONV_2D
13 MEAN
14 CONV_2D
15 SHAPE
16 STRIDED_SLICE
17 PACK
18 RESHAPE
\nHas GAP/MEAN/AVG POOL: True
Has CONV/FC classifier: True
Smoke output: [[-51  81]]
\n✅ TFLite looks TinyEngine-friendly (no broken [1,1] reshape)
Download: /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
Also on Drive: /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: TF 2.20: Please use the LiteRT interpreter from the ai edge litert package.
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

7b — Evaluate deployed INT8 TFLite on the test set

Section 6 reports **PyTorch FP32** accuracy. The MCU runs the **INT8 TFLite** from §7. This cell measures that artifact on the full test set and fails if it drifts more than 1% from PyTorch.

```
print('▶ INT8 TFLite full test-set evaluation', flush=True)

import numpy as np
import tensorflow as tf
import torch

assert os.path.exists(TFLITE_PATH), f'Missing TFLite: {TFLITE_PATH}'
assert 'te_acc' in dir(), 'Run Section 6 (PyTorch test eval) first'

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
inp = interp.get_input_details()[0]
out = interp.get_output_details()[0]
in_scale, in_zp = inp['quantization']
assert in_scale > 0, 'INT8 input missing quantization params'

def quantize_nhwk(chw_float: np.ndarray) -> np.ndarray:
    """ImageNet-normalized CHW float -> NHWC int8 (same affine as TFLite input)."""
    x = chw_float.transpose(1, 2, 0).astype(np.float32)
    v = x / float(in_scale)
    v = v + np.where(v == 0.0, 0.5, -0.5)
    q = np.trunc(v).astype(np.int32) + int(in_zp)
    return np.clip(q, -128, 127).astype(np.int8)

correct = 0
total = 0
class_correct = [0] * NUM_CLASSES
class_total = [0] * NUM_CLASSES
agree_pt = 0

model.load_state_dict(torch.load(SAVE_PATH, map_location='cpu'))
model.cpu().eval()

for i in range(len(test_dataset)):
    img, label = test_dataset[i] # CHW float32 ImageNet-normalized
    x_int8 = quantize_nhwk(img.numpy())
    interp.set_tensor(inp['index'], x_int8[None, ...])
    interp.invoke()
    y = interp.get_tensor(out['index']).reshape(-1)
    pred = int(np.argmax(y))

    with torch.no_grad():
        pt_pred = int(model(img.unsqueeze(0)).argmax(1).item())

    correct += int(pred == label)
    agree_pt += int(pred == pt_pred)
    total += 1
    class_total[label] += 1
    class_correct[label] += int(pred == label)

    if (i + 1) % 100 == 0 or (i + 1) == total:
        print(f' - {i+1}/{len(test_dataset)}', flush=True)

int8_acc = correct / total
pt_agree = agree_pt / total
gap = abs(int8_acc - te_acc)

print('=' * 65)
print('INT8 TFLite TEST results (deployed model)')
print('=' * 65)
print(f' INT8 accuracy : {int8_acc:.4f} ({int8_acc*100:.1f}%)')
print(f' PyTorch FP32 : {te_acc:.4f} ({te_acc*100:.1f}%) [Section 6]')
print(f' |Δ| accuracy : {gap:.4f} ({gap*100:.2f} pts)')
print(f' Pred agreement: {pt_agree:.4f} ({pt_agree*100:.1f}% same argmax as PT)')
for i, cls in enumerate(CLASSES):
    a = class_correct[i] / class_total[i] if class_total[i] else 0.0
    print(f' {cls:<10} {a:.3f} ({a*100:.1f}%) n={class_total[i]}')
print(f' Artifact : {TFLITE_PATH}')
print('=' * 65)

if gap > 0.01:
    raise RuntimeError(
        f'INT8 test acc ({int8_acc:.4f}) differs from PyTorch ({te_acc:.4f}) '
        f'by {gap:.4f} > 1%. Export pipeline is broken – do not deploy.'
    )
print('✅ INT8 = PyTorch test accuracy within 1%')
```

```
- 3628/3673
- 3629/3673
- 3630/3673
- 3631/3673
- 3632/3673
- 3633/3673
```

```

- 3632/3673
- 3634/3673
- 3635/3673
- 3636/3673
- 3637/3673
- 3638/3673
- 3639/3673
- 3640/3673
- 3641/3673
- 3642/3673
- 3643/3673
- 3644/3673
- 3645/3673
- 3646/3673
- 3647/3673
- 3648/3673
- 3649/3673
- 3650/3673
- 3651/3673
- 3652/3673
- 3653/3673
- 3654/3673
- 3655/3673
- 3656/3673
- 3657/3673
- 3658/3673
- 3659/3673
- 3660/3673
- 3661/3673
- 3662/3673
- 3663/3673
- 3664/3673
- 3665/3673
- 3666/3673
- 3667/3673
- 3668/3673
- 3669/3673
- 3670/3673
- 3671/3673
- 3672/3673
- 3673/3673
=====
INT8 TFLite TEST results (deployed model)
=====
INT8 accuracy : 0.9913 (99.1%)
PyTorch FP32 : 0.9910 (99.1%) [Section 6]
[Δ] accuracy : 0.0003 (0.03 pts)
Pred agreement: 0.9981 (99.8% same argmax as PT)
  leaf      0.994 (99.4%) n=1943
  not_leaf  0.988 (98.8%) n=1730
Artifact    : /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
=====
✅ INT8 == PyTorch test accuracy within 1%

```

8 — STM32F746NG budget check (vs ACLIS disease model)

Numbers for the **current disease model** come from `ACLIS_report_26thJune.docx` (26 June 2026). Leaf-gate flash is measured from the TFLite just written; SRAM/latency are engineering estimates until TinyEngine codegen.

```

1 import os

# — From ACLIS_report_26thJune.docx —
DISEASE = {
    'flash_kb': 723,          # INT8 model size
    'sram_kb': 271,          # runtime / peak activation memory
    'infer_ms': 797,         # STM32 inference time
    'acc_stm32': 0.840,
    'flash_budget_kb': 1024,
    'sram_budget_kb': 340,   # report table (chip has 320KB DTCM+SRAM; report uses 340)
}

tflite_kb = os.path.getsize(TFLITE_PATH) / 1024.0 if os.path.exists(TFLITE_PATH) else float('nan')

# Engineering estimates for TinyLeafGate @ 96x96 after TinyEngine
# (weights = tflite size; activations usually well under 100KB for this arch)
GATE = {
    'flash_kb': tflite_kb,
    'sram_kb_est': 80.0,    # refine after TinyEngine memory planner
    'infer_ms_est': 80.0,  # rough; 96x96 depthwise <= 176x176 MCUNet-in3
}

flash_headroom_now = DISEASE['flash_budget_kb'] - DISEASE['flash_kb']
sram_headroom_now = DISEASE['sram_budget_kb'] - DISEASE['sram_kb']

# Cascade: sequential invoke, shared activation arena sized for max(disease, gate)
flash_both = DISEASE['flash_kb'] + GATE['flash_kb']
sram_peak_shared = max(DISEASE['sram_kb'], GATE['sram_kb_est'])
sram_peak_naive = DISEASE['sram_kb'] + GATE['sram_kb_est'] # if separate arenas (avoid)

extra_flash = GATE['flash_kb']
extra_sram_shared = max(0.0, GATE['sram_kb_est'] - DISEASE['sram_kb']) # usually 0
extra_latency_when_leaf = GATE['infer_ms_est'] + DISEASE['infer_ms']
extra_latency_when_not = GATE['infer_ms_est'] # skip disease model

print('=' * 72)
print('STM32F746 resource picture')
print('=' * 72)
print('\n[Current disease model — ACLIS report]')
print(f'Flash (INT8)      : {DISEASE["flash_kb"]} KB / {DISEASE["flash_budget_kb"]} KB')
print(f'SRAM (runtime)     : {DISEASE["sram_kb"]} KB / {DISEASE["sram_budget_kb"]} KB')
print(f'Headroom flash     : {flash_headroom_now} KB')
print(f'Headroom SRAM      : {sram_headroom_now} KB')
print(f>Inference          : {DISEASE["infer_ms"]} ms')
print(f'Idle note          : TinyEngine buffers are typically statically reserved,')
print(f'                    so model SRAM footprint = {DISEASE["sram_kb"]} KB even when')
print(f'                    not actively inferring (plus LCD/camera frame buffers).')

print('\n[Leaf gate — this notebook]')
print(f'Flash (TFLite)      : {GATE["flash_kb"]} KB')
print(f'SRAM (estimate)     : ~{GATE["sram_kb_est"]} KB peak activations')
print(f'Latency (est.)      : ~{GATE["infer_ms_est"]} ms')

print('\n[Combined cascade on one STM32F746NG]')
print(f'Flash total         : {flash_both} KB / {DISEASE["flash_budget_kb"]} KB')
print(f'  ↳ ("FITS" ✅ if flash_both <= DISEASE["flash_budget_kb"] else "OVER ❌")')
print(f'Extra flash         : {extra_flash} KB')
print(f'Peak SRAM shared    : {sram_peak_shared} KB / {DISEASE["sram_budget_kb"]} KB')
print(f'  ↳ ("FITS" ✅ if sram_peak_shared <= DISEASE["sram_budget_kb"] else "OVER ❌")')
print(f'Extra peak SRAM     : {extra_sram_shared} KB (0 if gate <= disease arena)')
print(f'Avoid separate arenas: naive sum would be {sram_peak_naive} KB')
print(f'Latency if LEAF     : ~{extra_latency_when_leaf} ms (gate + disease)')
print(f'Latency if NOT      : ~{extra_latency_when_not} ms (gate only)')
print('=' * 72)

if flash_both > DISEASE['flash_budget_kb']:
    print('\n⚠️ Flash over budget. Reduce IMAGE_SIZE (e.g. 64), shrink channels, or')
    print('  replace disease MCUNet-in3 with a smaller net — do not widen TinyLeafGate.')
else:
    print('\n✅ Flash OK for dual-model deployment (weights only; firmware/UI still need room).')
    print('  Next: TinyEngine codegen on the TFLite, then wire gate before invoke() in main.cpp.')

=====
STM32F746 resource picture
=====

```

```
[Current disease model - ACLIS report]
Flash (INT8)      : 723 KB / 1024 KB
SRAM (runtime)   : 271 KB / 340 KB
Headroom flash   : 301 KB
Headroom SRAM    : 69 KB
Inference        : 797 ms
Idle note        : TinyEngine buffers are typically statically reserved,
                  so model SRAM footprint ≈ 271 KB even when
                  not actively inferring (plus LCD/camera frame buffers).

[Leaf gate - this notebook]
Flash (TFLite)   : 50.9 KB
SRAM (estimate)  : ~80 KB peak activations
Latency (est.)   : ~80 ms

[Combined cascade on one STM32F746NG1]
Flash total      : 773.9 KB / 1024 KB (FITS ✓)
Extra flash      : +50.9 KB
Peak SRAM shared : 271.0 KB / 340 KB (FITS ✓)
Extra peak SRAM  : +0.0 KB (0 if gate ≤ disease arena)
Avoid separate arenas: naive sum would be 351.0 KB
Latency if LEAF  : ~877 ms (gate + disease)
Latency if NOT   : ~80 ms (gate only)
=====
✓ Flash OK for dual-model deployment (weights only; firmware/UI still need room).
Next: TinyEngine codegen on the TFLite, then wire gate before invoke() in main.cpp.
```

9 — Next steps on the board

1. Download `aclis_leaf_gate_96x_full_int8.tflite` from Drive (`leaf_gate_output/`) into `Ikmal/Leaf Gate Model/` .
2. On your PC, from `Ikmal/Leaf Gate Model/` :

```
./tinyengine/venv/bin/python codegen_leaf_gate_c.py
```

Prefer official TinyEngine codegen if it succeeds on this fixed TFLite; otherwise the emitter fallback still works.

3. In STM32CubeIDE: refresh `ACLIS_IKMAL` , build, flash.
4. Test: non-leaf → gate message; leaf → disease scores.

Training metrics for the 2026-07-28 Colab run are in `TRAINING_RESULTS.md` .

The export above uses `GAP(keepdims) + 1x1 Conv` on purpose so TinyEngine does not hit the old broken Flatten/Reshape.

Colab paid products - Cancel contracts here